

ThreadZero — Mathematical Framework

MATHEMATICAL FRAMEWORK — NON-PROVISIONAL PATENT SPECIFICATION

TASK Format Structured Disclosure

Classification: Confidential — Patent Pending | Status: Non-Provisional Filing Annex | Format:
TASK-Structured Technical Specification

TASK X — MATHEMATICAL FRAMEWORK OF THREADZERO

TASK X.0 — OVERVIEW

[00X0]: ○○○○

TASK X.1 — NOTATION AND CORE OBJECTS

[00X1] Let the following definitions apply throughout this section:

- $\mathbf{K} \subseteq \{0,1\}^{256}$: keyspace of 32-byte x-only public keys.
- $\mathbf{V} \subseteq \mathbb{Z}_{\geq 0}$: fixed-point integer asset units.
- $\mathbf{T} : \mathbf{K} \rightarrow \mathbf{V}$: Merkle-Sum Sparse Merkle Tree (MS-SMT).
- $\mathbf{H}_{\text{root}} \in \{0,1\}^{256}$: MS-SMT root hash.
- $\mathbf{\Sigma}_{\text{root}} \in \mathbb{Z}_{\geq 0}$: MS-SMT root sum.
- $\mathbf{H}(\cdot)$: SHA-256 hash function.
- $\mathbf{P}_{\text{int}} \in \mathbb{G}$: internal Taproot key.
- $\mathbb{G}$: secp256k1 generator point.
- $\mathbf{q}$: secp256k1 curve order.

TASK X.2 — GAS-FREE TRANSFER METHOD (GFTM)

TASK X.2.1 — Pre-State

[00X2]: ○○○

V

S

= T(k

S

),

V

r

= T(k

r

)

Transfer validity condition:

V

S

≥ T

Root sum:

Σ

root

$= \Sigma$

$k \in K$

$T(k)$

TASK X.2.2 — Off-Chain State Mutation

[00x3] Define new tree T' as follows:

$T'(k) =$

V

s

$- T,$

if $k = k$

s

V

r

$+ T,$

if $k = k$

r

$T(k),$

otherwise

Vault Sum Invariant:

$$\begin{aligned} &\Sigma' \\ &\text{root} \\ &= \Sigma \\ &\text{root} \end{aligned}$$

This equation proves conservation of total asset quantity across any transfer operation. The total supply is invariant under all off-chain state mutations.

TASK X.2.3 — Commitment Hash

[00X4] Let H'_{root} be the MS-SMT root hash computed after state mutation.

Commitment:

$$\begin{aligned} C &= H(H' \\ &\text{root} \\ &) \end{aligned}$$

Scalar derivation:

$$t = \text{int}(C) \bmod q$$

TASK X.2.4 — Taproot Tweak

[00X5] Tweaked output key:

P

out

$= P$

int

$+ t \cdot G$

P_{out} is the **Pay-to-Taproot (P2TR)** output key anchoring all transfers within the epoch to the Bitcoin blockchain. The resulting commitment is publicly verifiable and cryptographically binding.

TASK X.3 — SOVEREIGN VERIFIER (SV)

TASK X.3.1 — Inputs

[00X6] The **Sovereign Verifier** receives the following inputs:

- Holder key: k_h
- Claimed amount: A_h
- Merkle proof: π
- Internal key: P_{int}
- On-chain Taproot key: P_{onchain}
- Physical quantity: Q_{phys}

TASK X.3.2 — Merkle Reconstruction

[00X7] Compute reconstructed root hash and sum:

$(\hat{H}$

```

root
,  $\Sigma$ 
root
) = MerkleSumReconstruct(k
h
, A
h
,  $\pi$ )

```

TASK X.3.3 — Vault Sum Invariant Check

[00x8] Verification requires the following condition to hold:

```

 $\Sigma$ 
root
= Q
phys

```

If this condition is false, a ***Solvency Error*** is raised, indicating that the claimed digital supply does not match the verified physical quantity.

TASK X.3.4 — Taproot Re-Derivation

[00x9] Compute commitment hash:

```

 $\hat{C} = H(\hat{H})$ 

```

```
root
```

```
)
```

Derive scalar:

```
 $\hat{t} = \text{int}(\hat{C}) \bmod q$ 
```

Reconstruct output key:

```
 $\hat{P}$ 
```

```
out
```

```
= P
```

```
int
```

```
+  $\hat{t} \cdot G$ 
```

Final equality check:

```
 $\hat{P}$ 
```

```
out
```

```
= P
```

```
onchain
```

If this condition is true, solvency and balance are cryptographically verified. The holder's claimed amount is proven correct relative to the on-chain anchor without revealing any other holder's data.

TASK X.4 — ELASTIC PHYSICAL PEG (EPP)

TASK X.4.1 — Fixed-Point Mapping

[00x10] Let:

- Q_{kg} : physical quantity in kilograms
- M : precision multiplier (e.g., $M = 1000$ for gram-level precision)

Digital supply:

$$\begin{aligned} Q & \\ \text{units} & \\ = \lfloor Q & \\ \text{kg} & \\ \cdot M \rfloor & \end{aligned}$$

Root sum:

$$\begin{aligned} \Sigma & \\ \text{root} & \\ = Q & \\ \text{units} & \end{aligned}$$

TASK X.4.2 — Elastic Supply Adjustment

[00x11] On audit event i:

Q

units

(i)

= | Q

kg

(i)

· M]

Supply delta:

Δ

(i)

= Q

units

(i)

– Q

units

(i–1)

- **Mint condition:** If $\Delta^{(i)} > 0$, mint $\Delta^{(i)}$ units.
- **Burn condition:** If $\Delta^{(i)} < 0$, burn $|\Delta^{(i)}|$ units.

Elastic Supply Invariant:

```
Σ  
root  
(i)  
= Q  
units  
(i)
```

This invariant guarantees that at every audit epoch, the total digital supply of ThreadZero tokens equals the verified physical quantity to fixed-point precision. No fractional reserve is permitted.

TASK X.5 — DUAL-PATH TAPROOT RECOVERY (DPTR)

TASK X.5.1 — Recovery Script

[00X12] Let:

- L : timelock parameter (block height delta)
- P_{rec} : recovery key

Recovery script semantics:

```
Script  
  
rec  
  
= CLTV(L) AND CHECKSIG(P
```

```
rec
```

```
)
```

Where **CLTV** denotes CheckLockTimeVerify enforced at the Bitcoin consensus layer, and **CHECKSIG** denotes a valid Schnorr signature verification against P_{rec} .

TASK X.5.2 — Taproot Construction

[00X13] Let H_{script} be the **MAST** (Merkelized Abstract Syntax Tree) root.

Taproot output:

```
P
```

```
out
```

```
= P
```

```
int
```

```
+ H
```

```
tweak
```

```
(P
```

```
int
```

```
, H
```

```
script
```

```
) ·  $\mathbb{G}$ 
```

Where H_{tweak} is the tagged hash function defined in BIP-341.

TASK X.5.3 — Recovery Condition

[00X14] Let h be the anchor block height at time of issuance. Recovery is valid if and only if **both** of the following conditions hold:

```
h
now
≥ h + L
VerifySig(P
rec
, σ, tx) = true
```

Where h_{now} is the current Bitcoin blockchain height, σ is the recovery signature, and tx is the recovery transaction. This construction enables deterministic, on-chain-enforced disaster recovery without reliance on any trusted third party.

— END OF TASK X — MATHEMATICAL FRAMEWORK —
